

Chapter 4

1. The lexical grammars of Python and Haskell are not regular. What does that mean, and why aren't they?

Python and Haskell do not have regular lexical grammars since they use indentation based blocks, which are untrackable using a finite automata.

2. Aside from separating tokens—distinguishing `print foo` from `printfoo`—spaces aren't used for much in most languages. However, in a couple of dark corners, a space *does* affect how code is parsed in CoffeeScript, Ruby, and the C preprocessor. Where and what effect does it have in each of those languages?

Whitespace affects how function calls and arguments are parsed in Coffeescript which means that whitespace can also be syntax. Whitespace can change how expressions are interpreted in Ruby which means a space can determine whether code is parsed as a method call or as an arithmetic expression. In C, whitespace affects macro expansion and tokenization before parsing occurs. Spaces can change how macros are expanded and how tokens are combined during preprocessing.

3. Our scanner here, like most, discards comments and whitespace since those aren't needed by the parser. Why might you want to write a scanner that does *not* discard those? What would it be useful for?

Whitespace and comments are extremely useful for refactoring, documentation comments, code formatting, syntax highlighting for errors, unused variables, undefined variables, etc.

4. Add support to Lox's scanner for C-style `/* ... */` block comments. Make sure to handle newlines in them. Consider allowing them to nest. Is adding support for nesting more work than you expected? Why?

```
case '/':
    if (match('/')) {
        while (peek() != '\n' && !isAtEnd()) advance();
    } else if (match('*')) {
        blockComment();
    } else {
        addToken(SLASH);
    }
    break;
```

```
private void blockComment() {
    int nesting = 1;

    while (nesting > 0 && !isAtEnd()) {
        if (peek() == '\n') {
            line++;
            advance();
        } else if (peek() == '/' && peekNext() == '*') {
            nesting++;
            advance();
            advance();
        } else if (peek() == '*' && peekNext() == '/') {
            nesting--;
            advance();
            advance();
        } else {
            advance();
        }
    }

    if (nesting > 0) {
        Lox.error(line, "Unterminated block comment.");
    }
}
```

This is definitely a lot more work than the original but I wouldn't say it's more work than I expected since the underlying problem is that we are looking for the beginning block syntax `/*` and then the ending block syntax `*/`. I feel like most of the work just comes from how much syntax jargon is in Java.

Chapter 5

1. Earlier, I said that the `|`, `*`, and `+` forms we added to our grammar metasyntax were just syntactic sugar. Take this grammar:

```
expr → expr ( "(" ( expr ( "," expr )* )? ")" | "." IDENTIFIER )+
      | IDENTIFIER
      | NUMBER
```

Produce a grammar that matches the same language but does not use any of that notational sugar.

```
expr → atom expr_tail
expr_tail → expr_fill
expr_tail → expr_fill expr_tail
expr_fill → "." IDENTIFIER
expr_fill → "(" arg ")"
```

```

arg → expr arg_tail
arg → ε
arg_tail → "," expr arg_tail
arg_tail → ε
atom → IDENTIFIER
atom → NUMBER

```

2. The Visitor pattern lets you emulate the functional style in an object-oriented language. Devise a complementary pattern for a functional language. It should let you bundle all of the operations on one type together and let you define new types easily.

```

data Shape = Shape {area :: Double, perimeter :: Double}

circle :: Double -> Shape
circle r = Shape {area = pi * r * r, perimeter = 2 * pi * r}

rectangle :: Double -> Double -> Shape
rectangle w h = Shape {area = w * h, perimeter = 2 * (w + h)}

```

3. In [reverse Polish notation](#) (RPN), the operands to an arithmetic operator are both placed before the operator, so **1 + 2** becomes **1 2 +**. Evaluation proceeds from left to right. Numbers are pushed onto an implicit stack. An arithmetic operator pops the top two numbers, performs the operation, and pushes the result. Thus, this:
 (1 + 2) * (4 - 3)
 in RPN becomes:
 1 2 + 4 3 - *
 Define a visitor class for our syntax tree classes that takes an expression, converts it to RPN, and returns the resulting string.

```

package com.craftinginterpreters.lox;

class RpnPrinter implements Expr.Visitor<String> {

    String print(Expr expr) {
        return expr.accept(this);
    }

    @Override
    public String visitBinaryExpr(Expr.Binary expr) {
        return expr.left.accept(this) + " " + expr.right.accept(this) + " " + expr.operator.lexeme;
    }

    @Override
    public String visitGroupingExpr(Expr.Grouping expr) {
        return expr.expression.accept(this);
    }

    @Override
    public String visitLiteralExpr(Expr.Literal expr) {
        if (expr.value == null) return "nil";
        return expr.value.toString();
    }

    @Override
    public String visitUnaryExpr(Expr.Unary expr) {
        String operator = expr.operator.lexeme;
        return expr.right.accept(this) + " " + operator;
    }
}

Run | Debug
public static void main(String[] args) {
    Expr expression = new Expr.Binary(
        new Expr.Grouping(
            new Expr.Binary(
                new Expr.Literal(1),
                new Token(TokenType.PLUS, "+", null, 1),
                new Expr.Literal(2)
            )
        ),
        new Token(TokenType.STAR, "*", null, 1),
        new Expr.Grouping(
            new Expr.Binary(
                new Expr.Literal(4),
                new Token(TokenType.MINUS, "-", null, 1),
                new Expr.Literal(3)
            )
        )
    );

    System.out.println(new RpnPrinter().print(expression));
}

```